Aho-Corasick – An Executive Summary

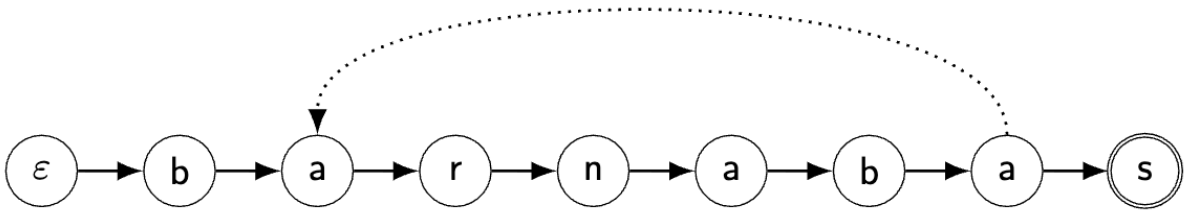
Introduction

Real world problems which necessitate simultaneous searching for multiple patterns within contiguous text include, but are not limited to, spam filtering, profanity filtering, network intrusion detection system, DNA sequencing, and even lexical analysis in compilers. Initially, an intuitive starting point for most people, where we have k unique substrings of length m in our dictionary, and an input size length n, we would have to search that text k times, once for each substring, to find all occurrences of all unique substrings within our dictionary. This naïve approach could be implemented with the Knuth-Moriss-Pratt (KMP) algorithm, which searches contiguous input text for one pattern. Given that this algorithm can only find the occurrence of one pattern within some input text, it would have to iteratively run the algorithm for as many times as there are words in a dictionary, making it O(k(m + n) for matching more than one pattern. A clear opportunity for optimizing this technique would be if there were an algorithm that could search for all substrings simultaneously. The ideal runtime would be O(m' + n + z) where m' is the total length of all patterns, n is the length of the text being searched, and z is the total number of matches found. This upgrade to KMP is known as the **Aho-Corasick Algorithm**.

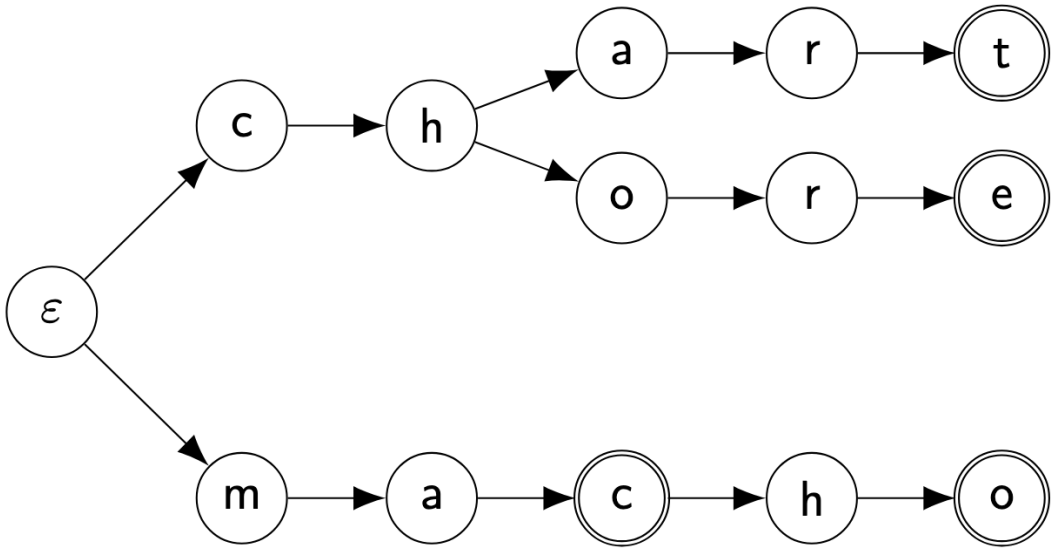
Overall Intuitive Approach

Let's start by understanding the Knuth-Morris-Pratt algorithm, which allows you to pass continuously through the corpus with no backtracking and no extra memory constraints while tracking your position in a DFA that searches for a specific search string in question. The key innovation is the concept of a failure link. There is a defined transition from each state, which represents the character subsequent to the current character in the search string. If you encounter any other character, the DFA transitions along the failure link without consuming the input character and runs again from the new location. This allows the DFA to keep track of what relevant prefix is prior to the current location in the corpus. After following the failure link, you check the input again and continue the process.

Below is an example of a DFA searching for the name Barnabas [case-insensitive]. I've highlighted the one relevant failure link from the third *'a'* to the first one. If at the last *'a'* the next character is not *'s'*, instead of transitioning all the way back to the epsilon at the start, it transitions back to the first *'a'* because the prefix *'ba'* is still intact. From other locations, the undrawn failure link goes all the way back to the epsilon.

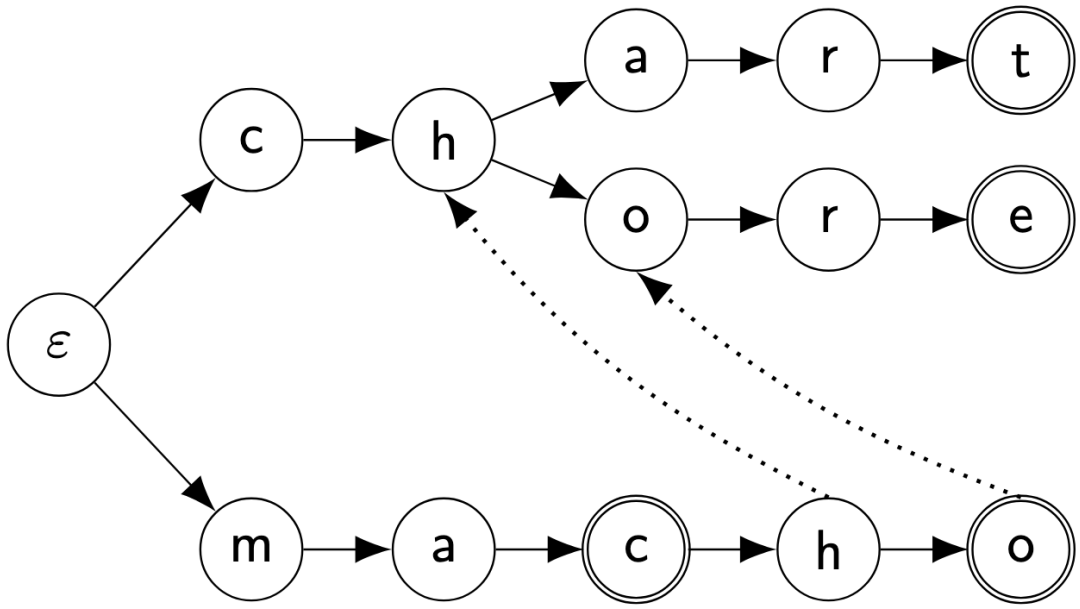


As covered in the introduction, we would rather be able to search for multiple strings simultaneously this way. A trie is a form of tree that allows us to check one query string against a whole dictionary of target strings using a similar DFA. If the transition that corresponds to the next character in the input exists from the current location in the DFA, the transition is performed. If not, there's been a failure and the string is clearly not in the dictionary. Here is what a trie looks like for the words 'chart', 'chore', 'mac', and 'macho'.



As you can see, terminal states are highlighted, and if the current state is a terminal state after processing the full input, the string is in the dictionary of the trie.

We can then add failure links to this trie the same way we did in KMP to get the internal representation we use for the Aho-Corasick algorithm. Every time we reach a terminal state, even if not at the end of our corpus, we mark that we have successfully found one of the substrings in our dictionary.



Implementation

The Aho-Corasick algorithm utilizes a deterministic finite automaton implemented through a 'trie' data structure and failure links. A 'trie' is a tree data structure used to store and retrieve strings. Its representation of a DFA comes from a trie's nodes serving as states and its edges serving as transitions. For the initial implementation of a trie, there must be as many leaf nodes as there are unique strings in a dictionary. In most implementations of a trie for Aho-Corasick, each node/vertex is what stores a boolean indicating a success or reject state, and a list of nodes which can be traversed to from the current node.

This means that for a multi-way trie, we can be searching for multiple words at the same time, given they share some prefix which is represented by diverging branches in a trie. Each branch starts with a transition/edge which holds the first letter to at least string in our dictionary represented by that branch. At that point, the next edge holds the next symbol of that specific dictionary string and connects to the next state. This continues until the leaf node of that branch is reached, indicating a successful match state, and we return that an occurrence for that specific string in our dictionary was matched. Although this basic trie can match any string in our dictionary, if the traversal of the first possible match doesn't end at a success node, we don't have a way to continue to check the rest of the input strings for any more matches.

```
# To add a string to the trie, we start at the root and for each character in the string,
# we check if there is already a vertex for that character. If not, we create a new
# vertex and link it to the current vertex. Finally, we mark the last vertex as an output AKA, the accept state.
# and store the full word there so we can print the match position later.
def add_string(s: str) -> None:
    v = 0
    # for every character in string
    for ch in s:
        # if we havent already made a vertex for this character, make one
        if ch not in t[v].next:
            # next[ch] is index of the next vertex we reach with character ch
            t[v].next[ch] = len(t)
            t.append(Vertex(p=v, pch=ch))
        # setting v to the next vertex in the trie
        v = t[v].next[ch]
    # this marks the final vertex of the string as an accept state
    t[v].output = True
    t[v].word = s
```

This is where we modify the basic structure of a trie, where if at any point during our traversal, we have a symbol which does not match the next symbol in the sequence of symbols within our given string; we do not progress and must restart at the root node. This is where the tree now begins to test if the failing symbol matches the first symbol of any strings in our dictionary once again. This is represented by edges/transitions known as failure links which connect every node, including the root, back to the root.

```
# This basically means that even after one pattern fails or succeeds, whereever it stops,
# we can look at the failure link to see if there's another pattern that can be matched from that
# point, using the same characters we already have
def get_link(v: int) -> int:
    # calculate failure link if the link hasnt been made yet
    if t[v].link == -1:
        # if its the root or direct child AKA the first character of a pattern
        # the link is 0 because the longest suffix that's also a prefix is the
        # empty string hence why it would just be the root
        if v == 0 or t[v].p == 0:
            t[v].link = 0
        else:
            # otherwise, get the failure link of the parent
            t[v].link = go(get_link(t[v].p), t[v].pch)
    return t[v].link
```

This now gives us a method to continuously search for any matches until the end of the input. However, for the trie to have the capability to search for multiple patterns simultaneously, we must reconstruct any failure links from any node whose longest suffix is also the prefix to some other node in our trie. This means that if the 3rd letter of branch one is an ‘A’ and which is also the first letter of branch two, if we are traversing down branch one, and we see that the fourth letter of input doesn’t match the next character in

branch one, we then take the failure link to the branch two and compare it to the second letter of its sequence.

```
# go computes the automaton transition for a character. If the trie already has an edge,
# we use it directly. Otherwise, we follow suffix links until we find the longest suffix
# that can still consume the character, or stay at the root if no such suffix exists.
def go(v: int, ch: str) -> int:
    if ch not in t[v].go:  # if we havent calculated this transition before
        if ch in t[v].next:  # if an edge already exist for this character
            t[v].go[ch] = t[v].next[ch]  # then we can just go to that vertex
        else:
            # assuming we're at the root, we stay if theres no character which is
            # the start of a pattern,
            # otherwise we follow the failure link and try to go from there
            t[v].go[ch] = 0 if v == 0 else go(get_link(v), ch)
    return t[v].go[ch]
```

Programming Challenge Summary

Our programming challenge focuses on the playful theme of censoring banned words from social media posts, because some sequences of characters are **so abhorrent that they should not be rendered in any direction**, **including forwards, backwards, or in a column including upside down**. (But if the banned sequence wraps across the line, it does not need to be censored.) Essentially, the words should not be found if you make a simple word search out of the corpus. We explain that the text will be rendered on a fixed-width monospace terminal, and there is a dictionary of sequences that need to be replaced with asterisk characters wherever encountered.

The learning objective of the programming assignment is to realize you have to locate all of the subsequences to be able to remove them, and you have to use a little bit of spatial reasoning to figure out how to ensure the subsequences aren't rendered in any "direction." The forward direction is easy enough and requires a plain implementation of Aho-Corasick where, after you pass through the character, if it turns out you have just found a subsequence, you must asterisk all the previous characters that were just in that subsequence.

Then, for the twist of getting it in multiple directions, you should learn how to reason through how the pattern renders on the fixed-width terminal, and how you can track multiple instances of the same DFA at once, where, for example, one is updated on each subsequent line in each column, tracking through the column.

We think this should not be too challenging, as once you have the main Aho-Corasick implementation done, you can keep an instance for every row and every column, and then, with fixed constant offsets, it's easy to refer back to the element one above in the column by going backwards w in the array or by keeping a 2D matrix.

Key Ideas for Solving the Programming Challenge

The key ideas are:

1. Realize that you need to use Aho-Corasick to identify the banned sequences of varying lengths.
2. Realize that once you identify the location of a banned sequence in Aho-Corasick, you must keep track of its position and range but not censor it case those characters overlap with another censored word from a different direction.
3. To account for backward sequences, you just add the reversed version of every banned word to the dictionary or iterate through the text in a different order.
4. The DFA created from Aho-Corasick can be reused because it is independent of the text being checked. Therefore, there are multiple ways to address this problem. One is running the DFA on the text four different times (one for each direction). Another is, instead of having one long-running instance of the DFA, you have one for each column and one for each row, and you update them only when you get to them. Because of the fixed shape of the text, you can do this easily with still only one pass through the main corpus.

We believe these challenges are the right difficulty level because they did not take us too long to come up with or implement, but they do require spatial reasoning and involve an interesting topic.

The solution should be able to be done in $O(m + n)$ time, where m is the dictionary size, n is the length of the corpus (in this case, the total length of all posts combined).

Conclusion

The Aho-Corasick algorithm is a natural extension of KMP that enables simultaneous multi-pattern searching in O(m + n + z) time compared to the naive O(k(m + n)) of running KMP once per pattern. Aho-Corasick achieves this by combining a trie, which encodes all patterns into a single DFA, with the failure links from KMP that allow the search to recover from mismatches without backtracking through the input.

The programming challenge applies this to a social media censorship problem, where banned character sequences must be masked in all four directions (forwards, backwards, vertical down, and vertical up) across a fixed-width matrix. The key insights are that Aho-Corasick handles multi-pattern detection efficiently; matches must be tracked before censoring to handle overlaps across different directions; reversed patterns can be handled by either reversing the banned word and adding it to the DFA or reversing the search order of the text; and the DFA can be reused across multiple rows and columns since it is independent of the text.

An example of two valid implementations for our programming challenge is four separate passes for each direction or a single pass with one DFA instance per row and column. Both produce correct results, with the single pass approach being more theoretically elegant while the multiple passes being simpler to implement and reason.

The target complexity for the programming challenge is O(m + n), where m is the total length of all patterns in the dictionary, and n is the total length of all posts. Since the problem is only concerned with tracking the maximum pattern length per state rather than all matches, z, the number of matches found is effectively O(n).

We hope you've enjoyed our report on the Aho-Corasick algorithm and how it can be understood clearly as a union of the Knuth-Morris-Pratt algorithm and the trie data structure. We hope that the programming assignment proves interesting to ponder and tackle, and that the experience overall proves educational.

Regards,

Lysander, Michael, Daniel.